

SPORT VS POLITICS TEXT CLASSIFICATION

Name: Ashutosh Ravi Gupta

Roll Number: P25CS0002

Github repo link: <https://github.com/Ashutosh0212/NLU/tree/main/Assignment%201>

1. Introduction

This report describes the design, implementation, and evaluation of a text classifier that automatically categorizes news articles as either **Sport** or **Politics**. The task falls under text classification in natural language processing (NLP), where the goal is to assign predefined labels to documents based on their content. We compare multiple machine learning techniques and feature representations to identify the best-performing approach for this binary classification problem.

2. Data Collection

2.1 Source

The dataset was derived from the **News Category Dataset v3** (HuffPost News), a publicly available collection of news articles. The original dataset is in JSON Lines (JSONL) format. Each record includes: *category*, *headline*, *short_description*, *authors*, *link*, and *date*.

Kaggle link : <https://www.kaggle.com/datasets/rmisra/news-category-dataset/data>

2.2 Extraction Process

A Python script (`extract_sport_politics.py`) was written to:

1. read the JSONL file line by line,
2. parse each line as JSON,
3. filter records where category equals "SPORTS" or "POLITICS",
4. extract headline and short_description,
5. combine them into a single text unit,
6. assign the corresponding label.

2.3 Class Balancing

The raw extraction yielded approximately 5,077 Sport articles and 35,601 Politics articles. We applied **undersampling** on the Politics class (random seed 42) to match the Sport count. The final balanced dataset contains **10,154 documents** (5,077 per class).

2.4 Storage Format

Data was stored as:

1. CSV file `sport_politics_balanced.csv` with columns text and label;
2. plain text files `data/sport.txt` and `data/politics.txt` with one document per line.

3. Dataset Description and Analysis

3.1 Description

This dataset contains around 210k news headlines from 2012 to 2022 from [HuffPost](#). This is one of the biggest news datasets and can serve as a benchmark for a variety of computational linguistic tasks. HuffPost stopped maintaining an extensive archive of news articles sometime after this dataset was first collected in 2018, so it is not possible to collect such a dataset in the present day. Due to changes in the website, there are about 200k headlines between 2012 and May 2018 and 10k headlines between May 2018 and 2022.

Content

Each record in the dataset consists of the following attributes:

- category: category in which the article was published.
- headline: the headline of the news article.
- authors: list of authors who contributed to the article.
- link: link to the original news article.
- short_description: Abstract of the news article.
- date: publication date of the article.

3.2 Size and Split

Metric	Value
Total documents	10,154
Sport documents	5,077
Politics documents	5,077
Train set (80%)	8,123
Test set (20%)	2,031

Random seed: 42.

3.3 Document Characteristics

Each document typically consists of a headline (5–15 words) followed by a short description (1–3 sentences). Total length ranges from about 20 to 150 words. All text is in English. Sport articles cover football, basketball, baseball, tennis, golf, Olympics, and athlete news. Politics articles cover government, elections, policy, international relations, and legislation.

4. Techniques in Brief

4.1 Feature Representations

These methods convert a document into a numerical vector so a computer can "read" it.

1. Bag of Words (BoW)

This treats a document like a literal bag of marbles. You throw all the words in and count how many of each there are.

- **Stop Words Removed:** Common words like "the," "is," and "at" are deleted because they don't carry unique meaning.
- **Top 10,000 Terms:** To keep the math from getting too heavy, we only track the 10,000 most frequent words across all documents.
- **The Flaw:** It ignores word order. "Dog bites man" and "Man bites dog" look identical to a BoW model.

2. TF-IDF (Term Frequency-Inverse Document Frequency)

BoW is often too simple because it rewards words just for being frequent. TF-IDF is smarter; it balances two things:

1. **Term Frequency (TF):** How often a word appears in *this* document.
2. **Inverse Document Frequency (IDF):** How unique a word is across *all* documents.

Example: In a set of 1,000 news articles, the word "election" might appear in every article (low IDF), but the word "platypus" appears in only one (high IDF). TF-IDF gives "platypus" more weight because it's a better "clue" for what that specific article is about.

3. N-grams (1,2)

This solves the "word order" problem of BoW. Instead of looking at single words (Unigrams), it also looks at pairs of consecutive words (Bigrams).

- **Unigram:** "Not," "good."
- **Bigram:** "Not good." By using N-grams, the model can capture context and sentiment that single words miss.

4.2 Machine Learning Classifiers

Once the text is turned into numbers, these algorithms do the sorting.

1. Naive Bayes (Multinomial)

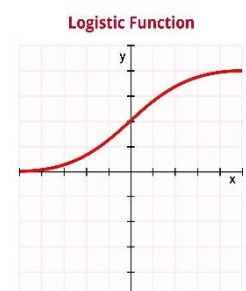
This is a "probabilistic" model. It calculates the odds of a document belonging to a class based on the words it contains.

- **"Naive":** It assumes every word is independent of the others. It doesn't care that "New" and "York" usually go together; it treats them as separate evidence.
- **Best for:** It is incredibly fast and works surprisingly well for spam filtering and basic sentiment analysis.

2. Logistic Regression

Despite the name, this is used for classification, not predicting a continuous number.

- **The Sigmoid Function:** It takes the input and squeezes it into a value between 0 and 1. If the result is > 0.5 , it's Class A; if < 0.5 , it's Class B.
- **Interpretable:** You can look at the "weights" to see exactly which words are pushing the decision toward a specific category.



3. Linear SVM (Support Vector Machine)

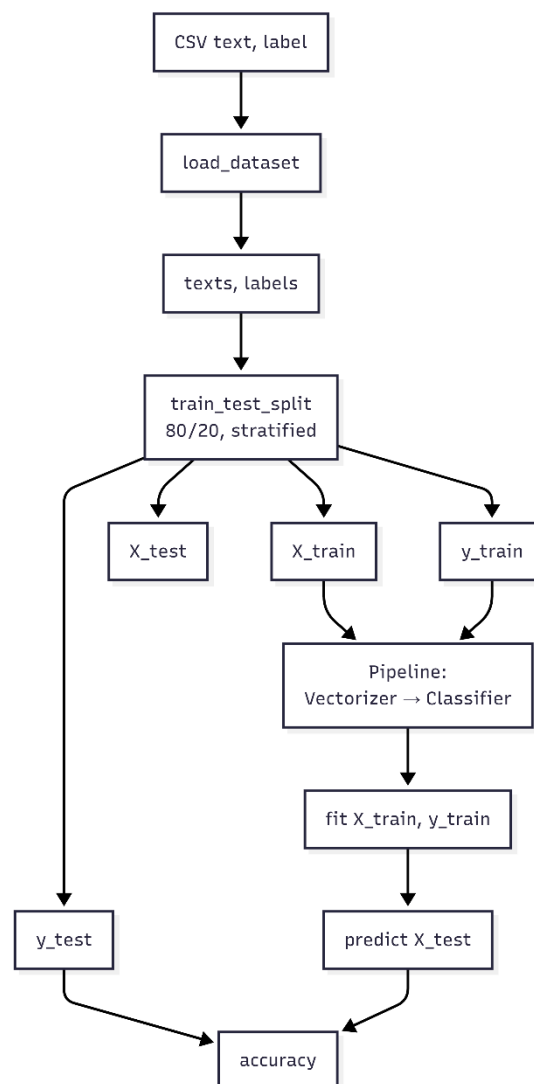
SVM tries to find the "widest possible street" (the margin) between two groups of data.

- **The Margin:** It doesn't just draw a line to separate classes; it positions the line so that it is as far away from the nearest data points as possible.
- **Robustness:** Because it focuses on the boundary between classes, it's very effective at handling "sparse" data (where most words in the vocabulary don't appear in every document).

4.3 Implementation

The classifier is implemented in a single Python script (P25CS0002_prob4.py) using scikit-learn. Raw text is loaded from CSV, converted into numerical features, and fed into one of three model configurations. The best model is selected by accuracy and used for interactive prediction.

4.3.1 Data Flow



Pipeline Structure

Each configuration is a scikit-learn Pipeline with two steps:

1. Vectorizer – converts raw text into a numeric feature matrix
2. Classifier – trains on the matrix and predicts labels

This ensures the vectorizer is fit only on training data and applied consistently to test data (no data leakage).

Config	Vectorizer	Classifier
1	CountVectorizer (BoW, 10k features, stop words)	MultinomialNB
2	TfidfVectorizer (10k features, stop words)	LogisticRegression
3	TfidfVectorizer (ngram_range=(1,2), 15k features)	LinearSVC

4.3.2 Main Steps

1. Load – Read CSV, keep rows with label in {sport, politics}.
2. Split – 80% train, 20% test, stratified by label, random_state=42.
3. Train & Evaluate – For each pipeline: fit(X_train, y_train), predict(X_test), compute accuracy.
4. Select Best – Sort by accuracy, take the top pipeline.
5. Interactive Mode – Retrain best pipeline on full data, then loop: read user input → predict([text]) → print SPORT or POLITICS.

5. Quantitative Comparisons

5.1 Accuracy Comparison

Configuration	Accuracy
BoW + Naive Bayes	0.9606 (Best)
N-grams(1,2) + Linear SVM	0.9567
TF-IDF + Logistic Regression	0.9517

5.2 Best Model: Detailed Metrics (BoW + Naive Bayes)

Class	Precision	Recall	F1-Score	Support
politics	0.9625	0.9587	0.9606	1016
sport	0.9588	0.9626	0.9607	1015
Accuracy			0.9606	2031

Both classes achieve similar precision and recall. No class bias.

5.3 Observations

Naive Bayes + BoW performed best because Sport vs Politics is well captured by unigram word distributions. TF-IDF did not outperform BoW in this setup. N-grams + SVM achieved competitive results, suggesting phrase-level features add useful signal.

6. Limitations of the System

- **Domain and source bias:** Dataset from HuffPost only; may not generalize to other publishers.
- **Binary classification only:** Articles spanning both domains are forced into one class.
- **Short text:** Documents are headline + brief description; limited context.
- **No semantic understanding:** Models rely on lexical patterns, not meaning or paraphrasing.
- **Temporal drift:** Performance may degrade over time without retraining.
- **Interpretability:** Exact reasoning for each prediction is not provided.
- **Computational:** For much larger corpora, more efficient implementations may be needed.

7. Conclusion

We built a Sport vs Politics classifier using the News Category Dataset v3. BoW + Naive Bayes achieved the highest accuracy (96.06%) with balanced performance across both classes. The system is suitable for news filtering and topic routing within similar domains. Limitations include generalization, semantic understanding, and handling of ambiguous documents. Future work could explore deep learning, cross-source evaluation, and multi-class classification.

References: News Category Dataset (HuffPost); scikit-learn (Pedregosa et al., JMLR 2011)